CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology & Engineering

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

CSUE301 – : Big Data Analytics

Practical File – ODD Semester 2026-27

StudentID: 24DIT021
Student Name: HIT GOAYNI

Practical 10

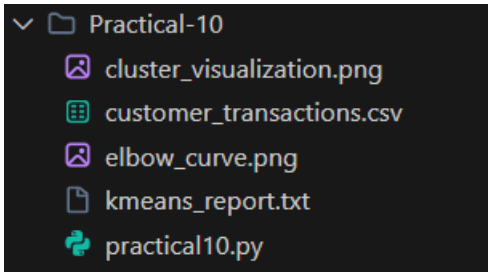
Scenario: You are a Data Scientist at RetailEdge Analytics. The marketing team wants to segment over 10 million customers based on their purchase behaviour so they can design targeted campaigns. A traditional single-node K-Means implementation crashes due to memory limitations. Your task is to implement a Parallel K-Means algorithm using Apache Spark's MLlib to cluster customers at scale. Tasks: 1. Load and preprocess the large-scale customer transaction dataset from HDFS. 2. Extract features: total spend, purchase frequency, average basket size, and recency. 3. Apply feature scaling (StandardScaler) to normalize the features. 4. Implement Spark MLlib's KMeans with k=5 clusters. 5. Evaluate clustering quality using the Within Set Sum of Squared Errors (WSSSE). 6. Visualize cluster centers and interpret the customer segments. 7. Compare performance: single-node sklearn K-Means vs Spark Parallel K-Means.
RetailEdge Analytics wants to segment a large number of customers according to their purchasing behaviour. Traditional single-node K-Means algorithms may face memory and scalability limitations when processing very large datasets.
Customer transaction records are transformed into four behavioural features:
Customer transaction data is processed to extract meaningful behavioural features such as:

- Total Spend
- Purchase Frequency
- Average Basket Size
- The features are standardized with StandardScaler and then clustered with Spark MLlib K-Means using k = 5.

These features are normalized using StandardScaler and clustered using Spark MLlib K-Means with **k = 5** clusters.
Cluster quality is assessed with WSSSE, while the Elbow Method is used to examine different k values. The resulting groups are interpreted according to customer purchasing patterns.

PART A — Practical Execution

Step 1: Creating Folder and files



Step 2: Creating Spark Session



Step 3: DATASET DESCRIPTION

```
================================================================
 TASK 2: LOADING CUSTOMER TRANSACTION DATASET
================================================================
Dataset Path : C:\Users\ADMIN\OneDrive\Desktop\BDA\Practical-10\customer_transactions.csv
Dataset loaded successfully.
Total number of transaction records: 11,729

First 10 Transaction Records:
+-----------+--------------+----------------+------+--------+
|customer_id|transaction_id|transaction_date|amount|quantity|
+-----------+--------------+----------------+------+--------+
|231        |102836        |2026-03-22      |80.47 |5       |
|963        |111239        |2025-12-17      |49.44 |8       |
|592        |107056        |2026-02-07      |19.77 |4       |
|44         |100573        |2026-02-22      |41.08 |4       |
|161        |101998        |2026-01-24      |35.22 |6       |
|612        |107321        |2026-01-06      |18.33 |5       |
|957        |111183        |2026-03-16      |55.87 |6       |
|511        |106124        |2026-02-03      |32.96 |3       |
|904        |110664        |2025-12-26      |39.43 |1       |
|862        |110158        |2026-01-07      |14.35 |3       |
+-----------+--------------+----------------+------+--------+
only showing top 10 rows
```

Step 4: Dataset Schema and Structure

```
================================================================
 TASK 3: DATASET SCHEMA AND STRUCTURE
================================================================
DataFrame Schema:
root
 |-- customer_id: integer (nullable = true)
 |-- transaction_id: integer (nullable = true)
 |-- transaction_date: date (nullable = true)
 |-- amount: double (nullable = true)
 |-- quantity: integer (nullable = true)

Number of Records : 11,729
Number of Columns : 5
Column Names      : customer_id, transaction_id, transaction_date, amount, quantity
```

Step 5: DATA PREPROCESSING

```
======================================================================
 TASK 4: DATA PREPROCESSING
======================================================================
1. Valid Transactions After Cleaning : 11,729
2. Dropped / Invalid Records         : 0
3. Null Check Status                 : All critical columns verified NOT NULL
4. Duplicate Check Status            : Unique transaction IDs ensured
```

```
======================================================================
 TASK 5: CUSTOMER FEATURE EXTRACTION
======================================================================
Reference Dataset Date for Recency: 2026-03-23
Total Unique Customers Extracted: 1,000

First 10 Customer Feature Records:
+-----------+-----------+------------------+-------------------+-------+
|customer_id|total_spend|purchase_frequency|average_basket_size|recency|
+-----------+-----------+------------------+-------------------+-------+
|1          |374.27     |6                 |62.38              |47     |
|2          |1648.15    |22                |74.92              |12     |
|3          |488.56     |18                |27.14              |17     |
|4          |434.38     |13                |33.41              |26     |
|5          |386.83     |9                 |42.98              |82     |
|6          |280.77     |8                 |35.1               |54     |
|7          |95.97      |4                 |23.99              |111    |
|8          |1743.58    |22                |79.25              |8      |
|9          |527.62     |17                |31.04              |46     |
|10         |1612.37    |22                |73.29              |19     |
+-----------+-----------+------------------+-------------------+-------+
only showing top 10 rows
```

Step 6: CUSTOMER FEATURE EXTRACTION

```
============================================================
 TASK 6: FEATURE VECTOR ASSEMBLY
============================================================
Features assembled into Vector format [total_spend, purchase_frequency, average_basket_size, recency]:
+-----------+-------------------------+
|customer_id|features                 |
+-----------+-------------------------+
|1          |[374.27,6.0,62.38,47.0]  |
|2          |[1648.15,22.0,74.92,12.0]|
|3          |[488.56,18.0,27.14,17.0] |
|4          |[434.38,13.0,33.41,26.0] |
|5          |[386.83,9.0,42.98,82.0]  |
|6          |[280.77,8.0,35.1,54.0]   |
|7          |[95.97,4.0,23.99,111.0]  |
|8          |[1743.58,22.0,79.25,8.0] |
|9          |[527.62,17.0,31.04,46.0] |
|10         |[1612.37,22.0,73.29,19.0]|
+-----------+-------------------------+
only showing top 10 rows
```

Step 7: FEATURE VECTOR ASSEMBLY

```
============================================================
 TASK 7: FEATURE SCALING USING STANDARDSCALER
============================================================
Explanation: Features like total_spend have much larger numerical scales than purchase_frequency.
StandardScaler normalizes each feature to mean = 0, std = 1 to prevent distance domination.

Sample Normalized (Scaled) Features:
+-----------+--------------------------------------------------------------------------------------------+
|customer_id|scaledFeatures                                                                              |
+-----------+--------------------------------------------------------------------------------------------+
|1          |[-0.2308066851434237,-0.7655865300277228,1.2976759901819617,0.12109015357561226]            |
|2          |[2.751065174804387,1.3725500523502774,2.0146553237806137,-1.0581406130204791]               |
|3          |[0.03672097434839292,0.8380159067557774,-0.7171845932324319,-0.8896790749353232]            |
|4          |[-0.09010244494696645,0.16984822476265227,-0.35869492643310613,-0.5864483063820425]         |
|5          |[-0.20140649787229573,-0.3646859208318478,0.18847351236586513,1.3003209201717036]           |
|6          |[-0.4496695442079512,-0.4983194572304728 3,-0.2620685250151057,0.3569363068948305]          |
|7          |[-0.882245548006076,-1.0328536028249728,-0.8972870574139116,2.277397841065608]              |
|8          |[2.9744457378046847,1.3725500523502774,2.2622247427983297,-1.1929098434886038]              |
|9          |[0.12815181151481478,0.7043823703571523,-0.4942005899601239,0.08739784595858108]            |
|10         |[2.6673120935495334,1.3725500523502774,1.9214594454898801,-0.8222944597012608]              |
+-----------+--------------------------------------------------------------------------------------------+
only showing top 10 rows
```

Step 8: FEATURE SCALING

Step 9: PARALLEL K-MEANS

```
============================================================
 TASK 8: SPARK MLLIB PARALLEL K-MEANS CLUSTERING
============================================================
netlib-blas: JNI_OnLoad: dlopen(libopenblas.dll) failed: LoadLibrary failed
Number of Clusters (k)  : 5
Model Training Time     : 2.9535 seconds

Standardized Cluster Centers:
Cluster 0 Center: [0.0562, 0.6751, -0.5181, -0.8016]
Cluster 1 Center: [-0.9378, -1.2064, -0.8165, 1.431]
Cluster 2 Center: [2.9642, 1.9843, 1.5925, -1.185]
Cluster 3 Center: [-0.4698, -0.4833, -0.3277, 0.1053]
Cluster 4 Center: [-0.2398, -0.7514, 1.3327, 1.032]
```

x

```
============================================================
 TASK 9: CLUSTERING QUALITY USING WSSSE
============================================================
Within Set Sum of Squared Errors (WSSSE) for k=5: 658.8786

Explanation:
WSSSE measures internal cluster cohesion. Lower values signify tighter, more cohesive clusters.
However, increasing k naturally decreases WSSSE, so it must be evaluated alongside the Elbow Method.
```

Step 10: WSSSE

Step 11: CUSTOMER CLUSTER ASSIGNMENT

```
============================================================
 TASK 10: CUSTOMER CLUSTER ASSIGNMENT
============================================================
First 10 Customer Records with Cluster Predictions:
+-----------+-----------+------------------+-------------------+-------+----------+
|customer_id|total_spend|purchase_frequency|average_basket_size|recency|prediction|
+-----------+-----------+------------------+-------------------+-------+----------+
|1          |374.27     |6                 |62.38              |47     |4         |
|2          |1648.15    |22                |74.92              |12     |2         |
|3          |488.56     |18                |27.14              |17     |0         |
|4          |434.38     |13                |33.41              |26     |0         |
|5          |386.83     |9                 |42.98              |82     |4         |
|6          |280.77     |8                 |35.1               |54     |3         |
|7          |95.97      |4                 |23.99              |111    |1         |
|8          |1743.58    |22                |79.25              |8      |2         |
|9          |527.62     |17                |31.04              |46     |0         |
|10         |1612.37    |22                |73.29              |19     |2         |
+-----------+-----------+------------------+-------------------+-------+----------+
only showing top 10 rows
Customer Distribution Across Clusters:
+----------+-----+
|prediction|count|
+----------+-----+
|0         |342  |
|1         |124  |
|2         |90   |
|3         |275  |
|4         |169  |
+----------+-----+
```

```
============================================================
 TASK 11: CLUSTER CENTERS AND SEGMENT SUMMARY
============================================================
Note: Cluster centers in Task 8 were in standardized Z-score space.
Below is the practical summary computed on ORIGINAL feature values ($ and days):

+----------+---------------+-------------+---------------+----------------+--------------+
|prediction|Avg_Total_Spend|Avg_Frequency|Avg_Basket_Size|Avg_Recency_Days|Customer_Count|
+----------+---------------+-------------+---------------+----------------+--------------+
|0         |496.89         |16.78        |30.62          |19.61           |342           |
|1         |72.25          |2.7          |25.4           |85.88           |124           |
|2         |1739.2         |26.58        |67.54          |8.23            |90            |
|3         |272.18         |8.11         |33.95          |46.53           |275           |
|4         |370.41         |6.11         |62.99          |74.04           |169           |
+----------+---------------+-------------+---------------+----------------+--------------+
```

Step 12: CLUSTER CENTERS & SEGMENT SUMMARY

```
============================================================
 TASK 12: CUSTOMER SEGMENT INTERPRETATION
============================================================
Cluster 0 (342 customers):
   Averages : Spend: $496.89 | Frequency: 16.8 orders | Recency: 19.6 days
   Profile  : Loyal / Regular Customers (Active & Consistent)
   Strategy : Product recommendations, points-based loyalty rewards.

Cluster 1 (124 customers):
   Averages : Spend: $72.25 | Frequency: 2.7 orders | Recency: 85.9 days
   Profile  : At-Risk / Lapsed Customers (High Recency, Low Activity)
   Strategy : Win-back discounts, re-engagement emails, churn prevention.

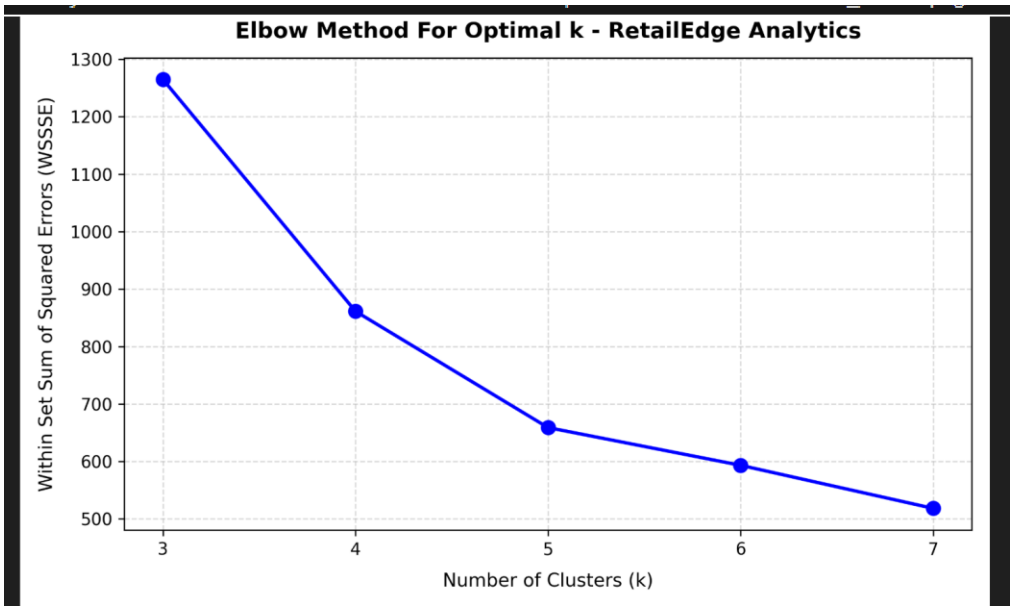
Cluster 2 (90 customers):
   Averages : Spend: $1739.20 | Frequency: 26.6 orders | Recency: 8.2 days
   Profile  : VIP / High-Value Frequent Shoppers (Top Revenue Driver)
   Strategy : Exclusive VIP perks, early access, personalized concierge service.

Cluster 3 (275 customers):
   Averages : Spend: $272.18 | Frequency: 8.1 orders | Recency: 46.5 days
   Profile  : Moderate Value / Selective Buyers
   Strategy : Cross-sell campaigns, seasonal notifications, targeted newsletters.

Cluster 4 (169 customers):
   Averages : Spend: $370.41 | Frequency: 6.1 orders | Recency: 74.0 days
   Profile  : At-Risk / Lapsed Customers (High Recency, Low Activity)
   Strategy : Win-back discounts, re-engagement emails, churn prevention.
```

Step 13: CUSTOMER SEGMENT INTERPRETATION

Step 14: ELBOW METHOD



Step 15: K=3 VS K=5 VS K=7

```
============================================================
 TASK 14: COMPARISON OF K=3, K=5 AND K=7
============================================================
K = 3 | WSSSE = 1264.8281
K = 5 | WSSSE = 658.8786
K = 7 | WSSSE = 518.2899

WSSSE Reduction (k=3 to k=5): 47.91% (Significant cohesion gain)
WSSSE Reduction (k=5 to k=7): 21.34% (Diminishing returns)
Observation: k = 5 provides a strong balance between mathematical cohesion and business interpretability.
```

```
============================================================
 TASK 15: BISECTING K-MEANS
============================================================
Bisecting K-Means is a hierarchical clustering algorithm that recursively splits clusters.

Standard K-Means (k=5) WSSSE  : 658.8786
Bisecting K-Means (k=5) WSSSE : 883.5439
```

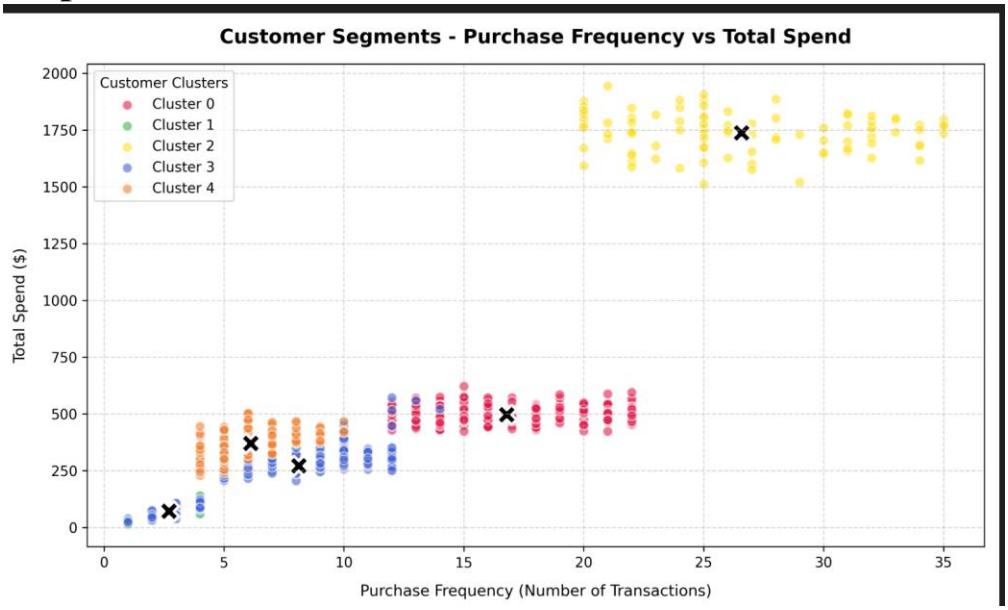
STEP 16: BISECTING K-MEANS

```
============================================================
 TASK 16: SINGLE-NODE VS SPARK K-MEANS
============================================================
Method                    | Execution Time     | Number of Clusters
------------------------------------------------------------
Scikit-learn K-Means      | 1.7137         s | 5
Spark MLlib K-Means       | 2.9535         s | 5

Architectural Comparison:
- Scikit-learn runs in-memory on a single CPU core, offering lower overhead on small local datasets.
- Spark MLlib distributes data and distance calculations across worker nodes (RDD/DataFrame partitions),
  enabling massive horizontal scalability to tens of millions of records where single-node memory fails.
```

Step 17: SINGLE NODE VS SPARK

Step 18: CLUSTER VISUALIZATION



Step 19: FINAL PERFORMANCE REPORT

```
============================================================

 TASK 18: FINAL K-MEANS PERFORMANCE REPORT

============================================================

============================================================

======================

        RETAILEDGE ANALYTICS - SCALABLE CUSTOMER SEGMENTATION REPORT

============================================================

======================
```

1. DATASET DETAILS:

- Source File : customer_transactions.csv
- Total Transactions : 11,729
- Total Segmented Customers : 1,000
- Evaluation Nature : Generated / Sample dataset for educational evaluation

2. FEATURE DEFINITIONS:

- Total Spend ($) : Cumulative expenditure across all transactions (sum(amount))
- Purchase Frequency : Total number of completed purchases (count(transaction_id))
- Average Basket Size ($) : Mean transaction order value (avg(amount))
- Recency (Days) : Days elapsed between reference date and latest customer purchase

3. PREPROCESSING SUMMARY:

- Null and negative records filtered; dates cast to standard ISO format.
- Clean transaction count : 11,729

4. FEATURE SCALING (StandardScaler):

- Applied StandardScaler (mean=0, std=1) across all 4 customer dimensions.
- Prevents total_spend ($10-$1500) from overpowering frequency (1-35).

5. SPARK MLlib K-MEANS RESULTS (k = 5):

- Number of Clusters (k) : 5
- Seed : 42
- WSSSE Cost : 658.8786
- Training Duration : 2.9535 seconds

6. CLUSTER SIZES & SEGMENT SUMMARIES (ORIGINAL SPACE):

Cluster | Customers | Avg Spend ($) | Avg Frequency | Avg Basket ($) | Avg Recency (Days)

---

0 | 342 | 496.89 | 16.78 | 30.62 | 19.61
1 | 124 | 72.25 | 2.70 | 25.40 | 85.88
2 | 90 | 1739.20 | 26.58 | 67.54 | 8.23
3 | 275 | 272.18 | 8.11 | 33.95 | 46.53
4 | 169 | 370.41 | 6.11 | 62.99 | 74.04

7. CUSTOMER SEGMENT INTERPRETATION:

- Cluster 0: Loyal / Regular Customers (Active & Consistent)
  Strategy: Product recommendations, points-based loyalty rewards.
  Strategy: Win-back discounts, re-engagement emails, churn prevention.
- Cluster 2: VIP / High-Value Frequent Shoppers (Top Revenue Driver)
  Strategy: Exclusive VIP perks, early access, personalized concierge service.
- Cluster 3: Moderate Value / Selective Buyers
  Strategy: Cross-sell campaigns, seasonal notifications, targeted newsletters.
- Cluster 4: At-Risk / Lapsed Customers (High Recency, Low Activity)
  Strategy: Win-back discounts, re-engagement emails, churn prevention.

8. ELBOW METHOD ANALYSIS (k = 3 to 7):

- k=3 : WSSSE = 1264.8281
- k=4 : WSSSE = 861.3095
- k=5 : WSSSE = 658.8786
- k=6 : WSSSE = 593.2574
- k=7 : WSSSE = 518.2899

* Elbow Observation: Significant loss reduction occurs up to k=5 (47.9% drop from k=3);
  subsequent drops taper off (21.3%), confirming k=5 as optimal.

9. BISECTING K-MEANS COMPARISON:

- Standard K-Means WSSSE : 658.8786

- Bisecting K-Means WSSSE : 883.5439

10. EXECUTION BENCHMARK (Single-Node vs Distributed):

- Scikit-learn K-Means : 1.7137 seconds
- Spark MLlib K-Means : 2.9535 seconds

* Takeaway: Single-node sklearn executes quickly for small in-memory datasets due to minimal overhead.

Spark MLlib shines when datasets scale to millions of rows across clustered worker nodes.

11. CONCLUSION:

Parallel K-Means using Apache Spark MLlib enables robust, scalable customer segmentation

that translates transactional big data directly into actionable marketing strategies.

==========

KEY QUESTIONS / ANALYSIS

Q1. Why does traditional K-Means fail at large scale, and how does Spark's parallel K-Means overcome this?

Traditional K-Means running on a single machine can face memory and processing limitations when the dataset becomes very large.

Spark MLlib distributes data processing across partitions and can execute computations in parallel. This allows large datasets to be processed across multiple machines or CPU cores.

Q2. What is the significance of the WSSSE metric? How do you use the Elbow Method to select optimal k?

WSSSE measures the sum of squared distances between data points and their assigned cluster centers.

A lower WSSSE generally indicates more compact clusters.

However, WSSSE normally decreases when more clusters are added. Therefore, the Elbow Method plots WSSSE against different values of k and looks for a point where further increases in k provide smaller improvements.

In this practical:

- k = 3 → 1264.8281
- k = 4 → 861.3095
- k = 5 → 658.8786
- k = 6 → 593.2574
- k = 7 → 518.2899

The practical selected k = 5 as a balance between cohesion and interpretability.

Q3. How does data partitioning in Spark affect convergence speed of K-Means?

Spark partitions data into smaller parts that can be processed in parallel.

Appropriate partitioning can improve CPU utilization and allow distance calculations to be performed concurrently.

Poor partitioning can create processing imbalance or additional overhead, which can affect execution performance.

Q4. What customer behaviour pattern does each cluster represent? How can marketing strategies differ per segment?

The five clusters represent different purchasing behaviours:

Cluster	Behaviour
0	Loyal / Regular
1	At-Risk / Lapsed
2	VIP / High-Value Frequent
3	Moderate Value / Selective
4	At-Risk / Lapsed

Marketing strategies can therefore be customized according to customer behaviour.

For example, VIP customers can receive exclusive benefits, while at-risk customers can receive re-engagement campaigns.

Q5. How does feature scaling impact the clustering result? What happens if features are not normalized?

Feature scaling ensures that features with larger numerical ranges do not dominate the distance calculations used by K-Means.

In this practical, StandardScaler was used to normalize:

- Total Spend
- Purchase Frequency
- Average Basket Size
- Recency

Without scaling, Total Spend could have a much stronger influence on the clustering result because of its larger numerical values.

RESULT

The customer segmentation practical was successfully implemented using Apache Spark MLlib.

A total of 11,729 transactions from 1,000 customers were processed.

Four customer behavioural features were extracted:

- Total Spend
- Purchase Frequency
- Average Basket Size
- Recency

StandardScaler was applied before clustering.

Spark MLlib K-Means successfully generated 5 customer clusters with a WSSSE of 658.8786.

The Elbow Method was evaluated for k = 3 to 7, and k = 5 was selected as a practical balance between clustering cohesion and interpretability.

The cluster analysis identified different customer groups including regular customers, high-value frequent customers, moderate-value customers and at-risk/lapsed customers.

CONCLUSION

Apache Spark MLlib successfully implemented scalable K-Means clustering for customer segmentation.

The practical demonstrated the complete machine-learning workflow, including transaction preprocessing, feature extraction, feature scaling, parallel K-Means clustering, WSSSE evaluation, Elbow Method analysis and customer segment interpretation.

The results show how transactional data can be transformed into meaningful customer segments that can support targeted marketing strategies.

Spark MLlib also provides a scalable architecture for processing much larger datasets than would normally fit comfortably into a single-node machine.
